Stokari · Granular Vertical Slices WBS

Model: 33 End-to-End Vertical Slices | **Total Tasks:** 188 Discrete Deliverables | **Baseline:** SDD v0.4 & SRS v0.5 | **Full-Stack Architecture:** UI + API + DB + Storage + Tests

Slice	Vertical Feature Domain	Tasks	Range	Full-Stack Subsystem Scope
VS-01	System Baseline & Dual Repository Toolchain	2	#1 – #2	Full-stack developer environments for Python 3.14 (Django + uv) and TypeScript (Vite+ + React 19).
VS-02	Contract-First Architecture & Kernel Primitives	8	#3 – #10	Innermost domain primitives (Result, DomainError, typed IDs) and the automated contract sync loop.
VS-03	Store Registration, Tenancy & Store Switcher	6	#11 – #16	Store onboarding, composite primary key tenancy, shop settings, store closure, and multi-store switcher UI.
VS-04	Google OIDC Authentication & Client Session Security	5	#17 – #21	Google OAuth token validation, custom ActorAuthentication, token rotation, and clock-rollback guards.
VS-05	Staff Management, Invites & Offline Leases	6	#22 – #27	Attendant invitations, lease expiration handling, instant deactivation, and persistent storage diagnostics.
VS-06	Money & Quantity Cross-Platform Math Engines	3	#28 – #30	Shared Python and TypeScript mathematical engines for integer santim, fractional milli-units, and rounding.
VS-07	Master Product Catalog & Table/Card Views	8	#31 – #38	Product model with composite PK, DRF viewsets, responsive DataTable (desktop) and card list (mobile).
VS-08	Product Packaging & Bulk Container Conversions	4	#39 – #42	Packaging definition rules (crates vs pieces), container multipliers, and the product creation wizard.
VS-09	Bilingual Amharic Search Normalization	4	#43 – #46	Ethiopic homophone folding (ሀ/ሓ/ኀ, ሰ/ሠ), sub-second in-memory search trie, and Noto Sans Ethiopic typography.
VS-10	POS Cashier Till & Staple Shortcuts	4	#47 – #50	Sub-10-second cashier till layout, 1-tap grocery staple shortcuts (Sugar, Rice, Oil, Coffee), and mobile segmented tabs.
VS-11	POS Cart, Keypad & Payment Selector	5	#51 – #55	Cart management, line total math, 3-button tender selector, decimal keypad sheet, and 54px thumb CTA.
VS-12	POS Durable Sales & Local Checkout (INV-01)	6	#56 – #61	Atomic Dexie transaction saving Sale + lines + outbox before UI confirmation, PostgreSQL sales schema, and immutability.
VS-13	Sale Cancellations & Compensating Ledger Audits	2	#62 – #63	Compensating SALE_CANCELLATION movements, reversesId foreign keys, idempotent double-cancels, and cancellation UI.
VS-14	Append-Only Stock Movement Ledger Core	5	#64 – #68	PostgreSQL StockChange table with immutability triggers, StockLogService.append with ascending row locks, and reconciliation tool.
VS-15	Client Stock Projections & Dexie UnitOfWork	5	#69 – #73	Dexie stock tables, local projection algorithm (confirmedQty + pendingDelta), atomic unit of work, and recovery rebuild.
VS-16	Stock Receiving & Supplier Intake	5	#74 – #78	Multi-line supplier shipment receiving, container-to-unit cost calculation with half-up rounding, and Receive Stock modal.
VS-17	Manual Stock Adjustments & Shrinkage	6	#79 – #84	Mandatory reason codes (Damage, Loss, Count Correction), delta adjustments, low-stock threshold badges, and summary cards.
VS-18	Stock Movement History & Audit Drawer	2	#85 – #86	Stock overview screen, movement history list, and slide-out audit drawer displaying cryptographic hashes.
VS-19	Offline Sync Server Push Pipeline & Idempotency	8	#87 – #94	POST /v1/sync/push, per-shop row locks, ProcessedOperation idempotency, ChangeLog sequence allocation, and outcome matrix.
VS-20	Client Outbox Dispatcher & Multi-Tab Web Lock	4	#95 – #98	Client outbox batch push (<=50 ops), exponential backoff with jitter, Web Lock coordinator, and ResultApplier.
VS-21	Offline Sync Pull Pipeline & Device Hydration	8	#99 – #106	GET /v1/sync/pull cursor-based deltas, initial bootstrap, CURSOR_RESET, reachability probe, and header SyncPill.
VS-22	AT-06 Conflict Detection & Review Queue	8	#107 – #114	StockBecameNegative event, ConflictCase model, Sell Anyway warning sheet, and owner review queue with parallel timeline.
VS-23	Conflict Arbitration & Owner Resolution Actions	7	#115 – #121	ResolveConflictHandler with optimistic locking, 3 resolution actions (Restock, Cancel, Acknowledge), and in-app notifications.
VS-24	Deactivated Staff Drain Flow & Security Quarantine	5	#122 – #126	10-minute drain token JWT, quarantine isolation (zero ledger impact), and owner Accept/Reject inspection tab.
VS-25	Today's Revenue Dashboard (3-Bucket Accounting)	6	#127 – #132	Offline Dexie aggregation grouping revenue into 3 mutually exclusive buckets (Confirmed, Pending, Review), and Today screen.
VS-26	Inventory Capital Valuation & Restock Alerts	3	#133 – #135	Store asset valuation calculation (sum max(qty, 0) x costSantim), uncosted item helper text, and restock warning cards.

Slice	Vertical Feature Domain	Tasks	Range	Full-Stack Subsystem Scope
VS-27	**Data Backup & Emergency Outbox Export**	3	#136 – #138	Streaming CSV exports, client emergency backup file generator (stokari-pending-export/v1.json), and export buttons.
VS-28	**Physical Inventory Count & Variance Reconciliation**	6	#139 – #144	CountSession model, shelf count entry sheet, server proposed variance accounting for intermediate sales, and COUNT_CORRECTION movement.
VS-29	**Standalone Local-Only Kiosk Mode**	4	#145 – #148	meta.mode = 'LOCAL_ONLY' toggle, persistent offline banner, export/import migration bundle, and transition documentation.
VS-30	**Design System Primitives & Responsive Shell**	8	#149 – #156	CSS variables (Cobalt #1E40AF), primitive component library with 44px targets, desktop sidebar, virtualized DataTable, and light-only audit.
VS-31	**Concurrency, Property & End-to-End Simulation**	9	#157 – #165	Multi-threaded race condition tests, Hypothesis property tests (1,000 runs), fast-check, Playwright two-device conflict simulation (AT-06).
VS-32	**Quality Gates, Accessibility & Traceability Enforcement**	13	#166 – #178	trace_check.py requirement gate, WCAG 2.1 AA axe-core audit, bundle size <=2MB, usability study with 5 users, and visual regressions.
VS-33	**Deferred Cloud Infrastructure & Disaster Recovery**	10	#179 – #188	Post-development cloud deployment: Render API, Vercel web, Neon cloud database, Sentry, and nightly backup restore tests.

Each vertical slice contains all tightly related tasks across frontend UI, backend API, database models/triggers, and automated tests, enabling any developer to ship working features end-to-end.

VS-01: System Baseline & Dual Repository Toolchain

Scope: Full-stack developer environments for Python 3.14 (Django + uv) and TypeScript (Vite+ + React 19). | **Count:** 2 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#1	VS-01-01	MUST	BACKEND / TOOLING	**Init stokari-api Django + uv, Ruff, mypy, import-linter, pytest, drf-spectacular** `backend/pyproject.toml, backend/backend/settings.py`	Python 3.14 environment locked with uv. Strict mypy type-checking, import-linter boundary gates, and drf-spectacular OpenAPI generator. **Deliverable:** uv run ruff check . and uv run python manage.py check pass with 0 errors.
#2	VS-01-02	MUST	FRONTEND / TOOLING	**Init stokari-web Vite+React+TS + Vitest, Playwright, dependency-cruiser, ESLint** `web/package.json, web/vite.config.ts, web/tsconfig.json`	React 19 + TypeScript strict mode with Vite+ unified toolchain (vp), Vitest test runner, and dependency-cruiser architecture rules. **Deliverable:** vp check and vp test execute cleanly.

VS-02: Contract-First Architecture & Kernel Primitives

Scope: Innermost domain primitives (Result, DomainError, typed IDs) and the automated contract sync loop. | **Count:** 8 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#3	VS-02-01	MUST	BACKEND / CORE	**common/kernel: Result, DomainError, typed IDs (ShopId, OpId, ProductId), Clock port, event bus** `backend/common/kernel/{types,result,errors,bus,clock}.py`	Pure Python zero-dependency primitives. Strongly typed NewType identifiers prevent ID swaps. Result[T, E] monad replaces unhandled exceptions. **Deliverable:** Unit tests pass with 100% test coverage.
#4	VS-02-02	MUST	BACKEND / HTTP	**common/http: RFC 9457 handler, request-id middleware, throttles, StrictIntegerField, pagination** `backend/common/http/{errors,middleware,serializers,pagination}.py`	RFC 9457 Problem Details error schema. Request ID correlation tracking. StrictIntegerField preventing float coercion in money and quantities. **Deliverable:** HTTP base classes and serializers pass unit tests.
#5	VS-02-03	MUST	BACKEND / PERSISTENCE	**common/persistence: TransactionManager (atomic + shop_lock), ShopScopedRepository, domain_event outbox** `backend/common/persistence/{manager,repository,outbox}.py`	ShopScopedRepository automatically scopes all database queries by shop_id. TransactionManager wraps multi-table writes under shop lock. **Deliverable:** Repository base classes verified with multi-tenant mock tests.
#6	VS-02-04	MUST	SHARED CONTRACT	**apps/shops/domain/permissions.py → contract/fixtures/permissions.json** `backend/apps/shops/domain/permissions.py, contract/fixtures/permissions.json`	Role-Permission matrix mapping OWNER vs ATTENDANT permissions across all operations, exported to a machine-readable JSON fixture. **Deliverable:** Fixture generation script produces valid permissions.json.
#7	VS-02-05	MUST	TOOLING / CONTRACT	**Contract bundle workflow: export_contract.py, contract-release.yml, contract.lock, pnpm contract:sync** `backend/scripts/export_contract.py, web/scripts/sync_contract.sh`	Automated contract synchronization loop pulling OpenAPI schema and generating TypeScript types in web/src/contract. **Deliverable:** Contract sync regenerates TypeScript types without manual edits.
#8	VS-02-06	MUST	SHARED FIXTURES	**Shared fixtures: money-vectors.json, quantity-vectors.json, payload-hash-vectors.json, permissions.json, error-codes.json, sample-operations/*.json** `contract/fixtures/*.json`	Committed canonical test vectors shared between backend Python and frontend TypeScript test suites. **Deliverable:** All shared fixtures committed and verified.
#9	VS-02-07	MUST	FRONTEND / CONTRACT	**Web contract tests: run every sample operation through Zod schemas; money/quantity vectors match Python** `web/src/contract/__tests__/contract-validation.test.ts`	Contract tests asserting frontend TypeScript schemas and vectors match Python backend outputs exactly. **Deliverable:** Vitest contract tests pass across all sample operations.
#10	VS-02-08	MUST	CI / QUALITY	**OpenAPI diff in CI: fails if openapi.json stale vs code** `.github/workflows/contract-diff.yml`	CI check verifying that openapi.json matches current code before merging pull requests. **Deliverable:** CI fails when API changes are not reflected in openapi.json.

VS-03: Store Registration, Tenancy & Store Switcher

Scope: Store onboarding, composite primary key tenancy, shop settings, store closure, and multi-store switcher UI. | **Count:** 6 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#11	VS-03-01	MUST	BACKEND / MODELS	**Django models: Shop, Membership, User, OfflineLease, RefreshToken, Invite, Device (composite PKs)** backend/apps/{shops,identity}/models.py	PostgreSQL schema using CompositePrimaryKey('shop', 'id') on all tenant tables. 5-attendant limit per shop. OfflineLease temporal windows. **Deliverable:** Initial Django migration (0001_initial.py) running cleanly on PostgreSQL.
#12	VS-03-02	MUST	BACKEND / DB	**Migrations 0001 (composite PKs) + 0002_protections (immutability triggers, composite FKs via RunSQL)** backend/apps/{shops,identity}/migrations/0002_protections.py	PostgreSQL RunSQL migration adding foreign keys referencing composite (shop_id, id) keys and database-level immutability triggers. **Deliverable:** Database migrations execute idempotently and enforce composite referential integrity.
#13	VS-03-03	MUST	BACKEND / API	**Endpoints: POST /shops (idempotency key), GET /shops/me, POST /shops/me/close (confirmation flow)** backend/apps/shops/views.py, backend/apps/shops/serializers.py	Store creation with idempotency key, store settings inspection, and confirmation-protected store closure with cascade lease revocation. **Deliverable:** Shop lifecycle endpoints tested with role permission assertions.
#14	VS-03-04	MUST	BACKEND / OPS	**Management commands: transfer_ownership, purge_closed_shop** backend/apps/shops/management/commands/{transfer_ownership,purge_closed_shop}.py	CLI tools for store ownership transfer and soft-deleted store data purging after 30-day grace period. **Deliverable:** Management commands pass dry-run and execution tests.
#15	VS-03-05	MUST	FRONTEND / UI	**Web: Multi-shop selection screen (if user has >1 membership)** web/src/features/shops/ShopSelectorScreen.tsx	Store selection interface allowing users with multiple memberships to pick their active store workspace. **Deliverable:** Shop selector transitions smoothly and updates active tenant context.
#16	VS-03-06	MUST	FRONTEND / UI	**Web: Create shop flow (first-time owner) + Accept invite flow (attendant)** web/src/features/onboarding/{CreateShopWizard,AcceptInviteScreen}.tsx	First-time owner store initialization wizard and attendant invite link redemption view. **Deliverable:** Onboarding flows complete end-to-end and initialize local database partition.

VS-04: Google OIDC Authentication & Client Session Security

Scope: Google OAuth token validation, custom ActorAuthentication, token rotation, and clock-rollback guards. | **Count:** 5 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#17	VS-04-01	MUST	BACKEND / AUTH	**AuthProvider + ActorAuthentication DRF class (verifies access JWT/drain token → Actor dataclass)** `backend/apps/identity/authentication.py`	Custom DRF authentication populating request.user with an immutable Actor dataclass. Completely bypasses django.contrib.auth production tables. **Deliverable:** Authentication tests asserting valid JWT resolves Actor and invalid token returns 401.
#18	VS-04-02	MUST	BACKEND / API	**Endpoints: GET /auth/nonce, POST /auth/google, POST /auth/refresh, POST /auth/lease, POST /auth/logout** `backend/apps/identity/views.py, backend/apps/identity/serializers.py`	Google ID token validation via google-auth library. PyJWT issuance for access/refresh pairs. Offline lease token generation (1-30 days duration). **Deliverable:** End-to-end API test suite covering full authentication handshake.
#19	VS-04-03	MUST	BACKEND / SECURITY	**Permission classes: ActiveMembership, ShopOpen, RolePermission (reads from fixture)** `backend/apps/shops/permissions.py`	DRF permission gates: ActiveMembership ensures user is not deactivated, ShopOpen ensures store is not closed, RolePermission validates action against permissions.json. **Deliverable:** Permission tests verifying owner-only actions return 403 to attendants (T-19).
#20	VS-04-04	MUST	FRONTEND / SESSION	**Web: session management (tokens in memory, refresh in IndexedDB, lease check, clock-rollback guard)** `web/src/session/{token-manager,lease-guard,clock}.ts`	Access token stored exclusively in memory (protecting against XSS). Refresh token in IndexedDB. Clock-rollback detector tracking meta.maxSeenTime. **Deliverable:** Session manager tests asserting clock tampering triggers immediate security lock.
#21	VS-04-05	MUST	FRONTEND / AUTH	**Web: Google Sign-In flow, Auth context (useAuth), authenticated route guards** `web/src/features/auth/{AuthContext,GoogleSignInButton,RequireAuth}.tsx`	React 19 context managing authentication state. Automatic silent refresh token rotation. Route guards redirecting unauthenticated users to login. **Deliverable:** Sign-in and session restore verified in browser integration tests.

VS-05: Staff Management, Invites & Offline Leases

Scope: Attendant invitations, lease expiration handling, instant deactivation, and persistent storage diagnostics. | **Count:** 6 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#22	VS-05-01	MUST	BACKEND / API	**Endpoints: POST /shops/me/invites, POST /invites/{token}/accept** backend/apps/shops/invites.py	Cryptographic invite token generator with 48-hour expiration. Atomic counter verification enforcing the 5-attendant quota under concurrency. **Deliverable:** Invitation generation and redemption verified with race condition test.
#23	VS-05-02	MUST	BACKEND / API	**Endpoints: GET /shops/me/members, POST /shops/me/members/{id}/deactivate** backend/apps/shops/members.py	Immediate attendant deactivation: invalidates active leases, revokes refresh tokens, and provisions a 10-minute drain token. **Deliverable:** Deactivation endpoint tested asserting immediate rejection on subsequent requests (AT-09).
#24	VS-05-03	MUST	FRONTEND / UI	**Web: Staff management screen (invite attendants, list members, deactivate)** web/src/features/staff/{StaffScreen,MemberList,InviteModal}.tsx	Owner UI showing active cashier leases, remaining days, invite link generator, and 1-tap attendant deactivation button. **Deliverable:** Staff screen verified matching stitch prototype layout and mobile responsiveness.
#25	VS-05-04	MUST	FRONTEND / STORAGE	**Web: navigator.storage.persist() requested at first run; quota warnings in Settings** web/src/data/persistence-manager.ts	Requests persistent storage permission from the browser to prevent automated cache eviction on Android Chrome and Safari. **Deliverable:** Storage persistence requested on app launch; returns granted status.
#26	VS-05-05	MUST	FRONTEND / UI	**Web: Persistent storage status shown in Settings (enabled/disabled)** web/src/features/settings/StorageGauge.tsx	Visual storage meter displaying used vs available disk space (navigator.storage.estimate()) with green Persistent Storage Active badge. **Deliverable:** Storage gauge renders accurately in staff_settings view.
#27	VS-05-06	MUST	FRONTEND / UI	**Web: Lease expiry UX — banner at 24h, read-only after expiry, export still works** web/src/features/auth/LeaseStatusBanner.tsx	Countdown alert when offline lease has <24 hours remaining. Disables sell buttons when expired while preserving emergency data export. **Deliverable:** Lease expiration states verified with mocked expired timestamps.

VS-06: Money & Quantity Cross-Platform Math Engines

Scope: Shared Python and TypeScript mathematical engines for integer santim, fractional milli-units, and rounding. | **Count:** 3 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#28	VS-06-01	MUST	SHARED CONTRACT	**Unit catalogue (apps/stock-log/domain/units.py) → contract/fixtures/units.json** backend/apps/catalog/domain/units.py, contract/fixtures/units.json	Canonical unit definitions (PIECE, KG, LITER, METER) with display symbols and decimal precision constraints exported to JSON contract. **Deliverable:** Export script generates versioned units.json fixture.
#29	VS-06-02	MUST	SHARED DOMAIN	**core/domain/quantity.py + core/domain/quantity.ts — parseQuantity, formatQuantity, validate_qty** backend/core/domain/quantity.py, web/src/core/domain/quantity.ts	Cross-language quantity math library using integer milli-units (1 kg = 1000 milli). Half-up rounding, bounds checking (0 to 1,000,000,000 milli). **Deliverable:** Both Python and TypeScript implementations pass identical test vectors (quantity-vectors.json).
#30	VS-06-03	MUST	SHARED SYNC	**Payload hash: RFC 8785 canonical JSON of {type, payload} → SHA-256 → base64url; shared vectors in contract/fixtures/payload-hash-vectors.json** backend/apps/sync/canonical_json.py, web/src/sync/canonical_json.ts	RFC 8785 canonical JSON serializer generating identical SHA-256 base64url hashes across Python and TypeScript. **Deliverable:** Both implementations pass shared payload-hash-vectors.json tests.

VS-07: Master Product Catalog & Table/Card Views

Scope: Product model with composite PK, DRF viewsets, responsive DataTable (desktop) and card list (mobile). | **Count:** 8 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#31	VS-07-01	MUST	BACKEND / MODELS	**Django Product model (composite PK, unit immutable, name/nameAlt, price/cost santim, lowStockMilli, status)** `backend/apps/catalog/models.py`	Product table with CompositePrimaryKey('shop', 'id'). Integer price_santim and cost_santim. Bilingual fields (name, name_alt). Status enum (ACTIVE, ARCHIVED). **Deliverable:** Product model migration running cleanly on PostgreSQL with composite indices.
#32	VS-07-02	MUST	BACKEND / DOMAIN	**CatalogService + ProductReader ports; ProductRepository (ShopScoped)** `backend/apps/catalog/{services,repositories,ports}.py`	Decoupled domain service and scoped repository. Enforces tenant isolation so shops cannot query or modify another store's products. **Deliverable:** Unit and integration tests asserting repository returns only shop-owned products.
#33	VS-07-03	MUST	BACKEND / HANDLERS	**Operations: CreateProduct (atomic with OPENING StockLog), UpdateProduct, SetProductStatus** `backend/apps/catalog/handlers.py`	Atomic product creation: if initial quantity > 0, atomically writes OPENING entry to StockChange ledger in the same database transaction. **Deliverable:** CreateProduct command handler verified emitting ledger entry and updating StockLevel.
#34	VS-07-04	MUST	FRONTEND / UI	**Web: Product detail (read-only unit badge, packaging editor [Stage 2])** `web/src/features/catalog/ProductDetailDrawer.tsx`	Slide-out inspector showing barcodes, retail prices, wholesale costs, margins, and packaging configurations. **Deliverable:** Product detail drawer renders with role-gated cost visibility.
#35	VS-07-05	MUST	FRONTEND / UI	**Web: Create/Edit Product modal (Zod validation mirroring backend)** `web/src/features/catalog/ProductFormModal.tsx`	React form with Zod schema validation mirroring backend DRF serializers. Live gross margin % calculator (price vs cost). **Deliverable:** Form validation catches invalid santim prices and negative stock thresholds.
#36	VS-07-06	MUST	FRONTEND / UI	**Web: Product card list (mobile) + DataTable (desktop) with density toggle** `web/src/features/catalog/ProductCatalogView.tsx`	Responsive presentation: high-density 8-column DataTable on desktop (>1024px) switching to tactile card stack on mobile (<1024px). **Deliverable:** Dual-view component renders cleanly at 390px and 1280px viewports.
#37	VS-07-07	MUST	FRONTEND / UI	**Product search toolbar: search input, category filter, status filter, 'Add Product' button** `web/src/features/catalog/CatalogToolbar.tsx`	Sticky search header with 1-tap category chips (Grains, Oils, Beverages) and quick-status filters (Low Stock, Out of Stock). **Deliverable:** Toolbar component matches stitch prototype styling and filter state.
#38	VS-07-08	SHOULD	SHARED DOMAIN	**Category management (CRUD) [Should]** `backend/apps/catalog/categories.py`, `web/src/features/catalog/CategoryModal.tsx`	Custom store categories allowing owners to organize their inventory taxonomy. **Deliverable:** Category creation and filtering verified end-to-end.

VS-08: Product Packaging & Bulk Container Conversions

Scope: Packaging definition rules (crates vs pieces), container multipliers, and the product creation wizard. | **Count:** 4 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#39	VS-08-01	MUST	FRONTEND / UI	**Web: Product-creation wizard ('How do you sell it?' → unit chips, preview sentence, 'Add another form')** `web/src/features/catalog/ProductWizard.tsx`	Intuitive onboarding wizard asking cashiers 'How do you sell this?' (By the piece, by weight in kg, or by volume in liters) with live preview. **Deliverable:** Product wizard modal verified with step transitions and packaging chips.
#40	VS-08-02	MUST	FRONTEND / COMPONENT	**Web: QuantityInput component — keypad by unit kind, quick chips from quickQtyMilli, g/mL toggle for KG/L** `web/src/ui/QuantityInput.tsx`	Unit-aware quantity input component rendering integer stepper for PIECE or decimal keypad with g/kg toggle for KG. **Deliverable:** QuantityInput verified across all 4 unit types with touch event tests.
#41	VS-08-03	SHOULD	BACKEND / PACKAGING	**Packaging rules [Should]: Product.packagings[], ReceiveLineInput.packaging, StockChange.packaging snapshot** `backend/apps/catalog/packaging.py`	Packaging definition schema supporting bulk containers (e.g., Crate of 24, Carton of 12) with conversion multiplier to base units. **Deliverable:** Packaging models and serializers pass conversion calculation tests.
#42	VS-08-04	SHOULD	BACKEND / HANDLERS	**Opening stock supports openingPackaging (count × factor)** `backend/apps/catalog/handlers.py`	Enables entering opening inventory as '5 crates' which automatically records as '120 pieces' in the base ledger. **Deliverable:** Opening stock calculation verified with container multiplication tests.

VS-09: Bilingual Amharic Search Normalization

Scope: Ethiopic homophone folding (ሀ/ሐ/ኀ, ሰ/ሠ), sub-second in-memory search trie, and Noto Sans Ethiopic typography. | **Count:** 4 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#43	VS-09-01	MUST	FRONTEND / SEARCH	**core/search/normalize.ts — Latin + Ethiopic homophone folding (ሀ/ሐ/ኀ, ሰ/ሠ)** `web/src/core/search/normalize.ts`	High-speed string normalizer mapping Ethiopic homophone glyphs to canonical forms. Enables searching 'ስኳር' or 'ሰክዋር' interchangeably. **Deliverable:** Benchmarked search normalizer folding 2,000 product titles in < 2ms.
#44	VS-09-02	MUST	FRONTEND / UI	**Web: Product list/search (in-memory normalized index, Amharic fold, virtualization)** `web/src/features/catalog/ProductSearchIndex.ts`	In-memory trie search index querying 2,000 cached products in under 50ms on mobile hardware, supporting Latin and Amharic queries. **Deliverable:** Search latency benchmark < 50ms on simulated low-end mobile CPU.
#45	VS-09-03	SHOULD	SHARED I18N	**i18n: all strings in en.json from day one; Amharic am.json [Should]** `web/src/i18n/{en,am}.json, web/src/i18n/index.ts`	Bilingual string dictionary mapping all UI labels to English and Amharic with zero hardcoded English strings in components. **Deliverable:** Language switcher toggles UI strings without layout distortion.
#46	VS-09-04	MUST	FRONTEND / TYPOGRAPHY	**Amharic text rendering: no baseline shift, proper font fallback (Noto Sans Ethiopic)** `web/src/index.css`	Font stack configuration ensuring Amharic glyphs render with consistent vertical metrics and zero character clipping. **Deliverable:** Amharic text renders cleanly across Chrome, Safari, and Firefox.

VS-10: POS Cashier Till & Staple Shortcuts

Scope: Sub-10-second cashier till layout, 1-tap grocery staple shortcuts (Sugar, Rice, Oil, Coffee), and mobile segmented tabs. | **Count:** 4 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#47	VS-10-01	MUST	FRONTEND / UI	**Web: Sell screen — search → tap product → quantity chips → payment label → Save (≤10s, ≤3 taps)** web/src/features/pos/SellScreen.tsx	High-velocity cashier till layout achieving sub-10-second checkout flow. Mobile segmented tab toggle between Catalog and Cart. **Deliverable:** Sell screen verified matching stitch prototype layout and mobile responsiveness.
#48	VS-10-02	MUST	FRONTEND / COMPONENT	**Web: Product quick chips (horizontal scroll, max 6 frequent items)** web/src/features/pos/QuickFavoritesBar.tsx	Top-row 1-tap quick buttons for frequent grocery staples (Sugar, Rice, Oil, Coffee) with 44px touch targets. **Deliverable:** Quick chips add product to cart with a single tap.
#49	VS-10-03	MUST	FRONTEND / STORAGE	**Web: Recent products persisted in meta.recentProducts** web/src/data/recent-products.ts	Local storage tracking top 6 cashier-selected items across browser restarts without server network calls. **Deliverable:** Recent items persist in IndexedDB meta table across page refreshes.
#50	VS-10-04	MUST	FRONTEND / POLISH	**Mobile Sell screen: header, search, quick chips, cart, stepper, payment selector, complete sale button** web/src/features/pos/SellScreenMobile.tsx	Mobile-optimized cashier POS layout verified with 44px+ touch targets and zero horizontal overflow. **Deliverable:** Mobile POS verified on 375px and 390px viewport viewports.

VS-11: POS Cart, Keypad & Payment Selector

Scope: Cart management, line total math, 3-button tender selector, decimal keypad sheet, and 54px thumb CTA. | **Count:** 5 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#51	VS-11-01	MUST	FRONTEND / UI	**Web: Cart — line qty unit × price = lineTotal, rounded sum, 'Sell anyway' sheet when exceeding projected stock** web/src/features/pos/CartPanel.tsx	Cart manager calculating line totals, applying half-up santim rounding, and triggering stock warning if cart exceeds available quantity. **Deliverable:** Cart calculations pass rounding tests; shows Sell Anyway alert on oversell.
#52	VS-11-02	MUST	FRONTEND / COMPONENT	**Web: Payment Selector — segmented control (Cash/Telebirr/Other)** web/src/features/pos/PaymentSelector.tsx	High-contrast 3-button tender label selector (Cash, Telebirr, Other). Label only, zero payment settlement integration. **Deliverable:** Payment selector tested with keyboard navigation and touch feedback.
#53	VS-11-03	MUST	FRONTEND / COMPONENT	**Web: Complete Sale button (54px sticky footer mobile, auto-width desktop)** web/src/features/pos/CheckoutFooter.tsx	Full-width 54px thumb-zone CTA bar on mobile viewports displaying live total and 1-tap checkout trigger. **Deliverable:** Checkout button renders in thumb zone without colliding with OS home bar.
#54	VS-11-04	MUST	FRONTEND / UI	**Web: Quantity Keypad Sheet (40vh, numeric/decimal by unit, quick chips, live total preview)** web/src/features/pos/QuantityKeypadSheet.tsx	Slide-up touch keypad optimized for entering arbitrary decimal weights (e.g. 2.750 kg) with standard increment chips (+0.25, +0.5, +1.0 kg). **Deliverable:** Keypad sheet verified with decimal entry and live total calculation.
#55	VS-11-05	MUST	FRONTEND / POLISH	**Mobile Quantity Keypad Sheet (numeric/decimal by unit, quick chips, live total)** web/src/features/pos/QuantityKeypadMobile.tsx	Slide-up touch keypad verified for one-handed operation on mobile devices. **Deliverable:** Keypad transitions smoothly with spring physics and zero tap lag.

VS-12: POS Durable Sales & Local Checkout (INV-01)

Scope: Atomic Dexie transaction saving Sale + lines + outbox before UI confirmation, PostgreSQL sales schema, and immutability. | **Count:** 6 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#56	VS-12-01	MUST	BACKEND / MODELS	**Django Sale, SaleLine, SaleCancellation models (composite PKs, reversesId on cancellation)** backend/apps/sales/models.py	Sale and SaleLine tables using CompositePrimaryKey('shop', 'id'). ReversesId foreign key on SaleCancellation pointing to original sale. **Deliverable:** Sales models migration running cleanly on PostgreSQL with composite FKs.
#57	VS-12-02	MUST	BACKEND / HANDLERS	**RecordSaleHandler: validates lines, recomputes totals, price snapshots, builds SALE entries, calls StockLogService.append, applies outcome matrix by mode** backend/apps/sales/handlers/record_sale.py	Validates item lines, recomputes total santim, snapshots unit prices, and appends negative movements to the stock ledger atomically. **Deliverable:** RecordSaleHandler unit tests pass with price snapshot assertions.
#58	VS-12-03	MUST	FRONTEND / USECASE	**Web recordSale.ts (offline: one Dexie tx → sale, lines, stock log, projection, outbox; online: outbox only, await confirmation)** web/src/features/pos/usecases/recordSale.ts	INV-01 Durable Local Save: commits Sale, SaleLines, pending StockChange, and Outbox entry in a single atomic Dexie transaction before returning success. **Deliverable:** Integration test verifying tab kill or crash immediately after save preserves record (T-07).
#59	VS-12-04	MUST	FRONTEND / UI	**Web: Pending confirmation list (online) + 'Awaiting confirmation' badge** web/src/features/pos/PendingSalesList.tsx	Visual list of recent sales displaying 'Awaiting Confirmation' while sync outbox processes the batch. **Deliverable:** Pending badge transitions to Synced once server confirms.
#60	VS-12-05	MUST	FRONTEND / UI	**Web: Sale history (localDay EAT grouping, sync state badges)** web/src/features/sales/SaleHistoryScreen.tsx	Sales journal grouping transactions by calendar date in Africa/Addis_Ababa timezone with status badges (Synced, Pending, Review). **Deliverable:** Sale history renders with proper timezone grouping and receipt preview.
#61	VS-12-06	MUST	BACKEND / TEST	**Sale immutability: architecture test — UPDATE/DELETE on sale/stock log/cancellation fails** backend/tests/architecture/test_immutability.py	Automated architectural test proving SQL UPDATE and DELETE statements on sales tables raise database integrity errors. **Deliverable:** Architecture test passes against PostgreSQL database.

VS-13: Sale Cancellations & Compensating Ledger Audits

Scope: Compensating SALE_CANCELLATION movements, reversesId foreign keys, idempotent double-cancels, and cancellation UI. | **Count:** 2 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#62	VS-13-01	MUST	BACKEND / HANDLERS	**CancelSaleHandler: inserts SaleCancellation (PK (shopId, saleId)), builds SALE_CANCELLATION entries with reversesId; quarantines on CANCELLATION_CONFLICT conditions** backend/apps/sales/handlers/cancel_sale.py	Appends compensating positive ledger movements. Idempotent on second cancellation attempt. Quarantines if sale was already cancelled. **Deliverable:** CancelSaleHandler verified asserting single compensating movement (AT-08).
#63	VS-13-02	MUST	FRONTEND / USECASE	**Web cancelSale.ts (owner only, offline pending)** web/src/features/pos/usecases/cancelSale.ts	Client cancellation use case: flags local sale as cancelled, updates local projected stock, and stages cancellation op in outbox. **Deliverable:** Cancel sale use case passes with outbox stage assertions.

VS-14: Append-Only Stock Movement Ledger Core

Scope: PostgreSQL StockChange table with immutability triggers, StockLogService.append with ascending row locks, and reconciliation tool. | **Count:** 5 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#64	VS-14-01	MUST	BACKEND / MODELS	**Django StockLevel + StockChange models (composite PK, immutability trigger, underReview flag)** backend/apps/ledger/models.py	Append-only StockChange table and StockLevel projection table with CompositePrimaryKey('shop', 'id'). under_review flag for conflict tracking. **Deliverable:** Ledger models migration running cleanly on PostgreSQL.
#65	VS-14-02	MUST	BACKEND / SERVICE	**StockLogService.append(entries, ctx) — locks StockLevel rows, increments version, writes StockChange, allocates serverSeq, returns crossed_below_zero** backend/apps/ledger/services/append.py	Core inventory append engine acquiring row locks in ascending product ID order. Increments projection version and allocates monotonic serverSeq. **Deliverable:** LedgerService passes concurrency tests asserting zero lost updates under load.
#66	VS-14-03	MUST	BACKEND / QUERIES	**StockLogQueries: quantity, history, balance_at_version(baseVersion)** backend/apps/ledger/queries.py	Optimized read queries retrieving current balance, movement history, and historical balance at specific projection versions. **Deliverable:** Query functions pass performance benchmarks on 50,000 movements.
#67	VS-14-04	MUST	BACKEND / OPS	**Nightly reconcile_stock_log management command (verifies qty = SUM(delta), version = COUNT)** backend/apps/ledger/management/commands/reconcile_stock_log.py	Automated verification job asserting StockLevel.quantity == SUM(StockChange.delta) across all products in the database. **Deliverable:** Command executes and reports zero discrepancies.
#68	VS-14-05	MUST	BACKEND / API	**Stock history endpoint: GET /products/{id}/stock-changes (role-filtered, paged)** backend/apps/ledger/views.py	Paginated stock movement history endpoint with role-based field filtering (stripping wholesale costs for attendants per NFR-12). **Deliverable:** Endpoint tests verify attendants receive movement counts with cost fields removed.

VS-15: Client Stock Projections & Dexie UnitOfWork

Scope: Dexie stock tables, local projection algorithm (confirmedQty + pendingDelta), atomic unit of work, and recovery rebuild. | **Count:** 5 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#69	VS-15-01	MUST	FRONTEND / STORAGE	**Web Dexie schema: stockChanges, stockLevels (projection with confirmedQty, pendingDelta, projectedQty)** web/src/data/schema/stock.ts	IndexedDB tables tracking confirmed server movements, confirmed stock levels, and computed local projected balances. **Deliverable:** Dexie schema version 1 initializes and passes table creation tests.
#70	VS-15-02	MUST	FRONTEND / DATA	**Web StockLogRepository.appendPending(), recomputeFor(productIds) — projection algorithm (§5.10)** web/src/data/repositories/StockLogRepository.ts	Client stock projection algorithm: projectedQty = confirmedQty + pendingDelta with zero double-counting of in-flight outbox operations. **Deliverable:** Projection algorithm passes fast-check property tests.
#71	VS-15-03	MUST	FRONTEND / DATA	**Web UnitOfWork (Dexie transaction wrapping sale + lines + stock log + outbox)** web/src/data/UnitOfWork.ts	Client atomic transaction wrapper ensuring multi-table consistency across sales, movements, and outbox tables in Dexie. **Deliverable:** UnitOfWork tests assert rollback on simulated IndexedDB write failure.
#72	VS-15-04	MUST	FRONTEND / RECOVERY	**Web recovery: rebuildProjection(), integrity check (local confirmed = sum of confirmed entries)** web/src/data/recovery.ts	Disaster recovery routine rebuilding local stock projection tables from raw movement records if IndexedDB is interrupted. **Deliverable:** Recovery routine passes state corruption test.
#73	VS-15-05	MUST	FRONTEND / COMPONENT	**Web: StockLevelDisplay component — confirmed vs projected, low-stock threshold, unit badge** web/src/ui/StockLevelDisplay.tsx	Visual badge component displaying confirmed balance alongside pending outbox deltas. **Deliverable:** Badge renders confirmed vs projected numbers with status colors.

VS-16: Stock Receiving & Supplier Intake

Scope: Multi-line supplier shipment receiving, container-to-unit cost calculation with half-up rounding, and Receive Stock modal. | **Count:** 5 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#74	VS-16-01	MUST	FRONTEND / UI	**Web: Receive Stock modal (packaging selector with live '3 crates = 36 pcs' preview)** `web/src/features/stock/ReceiveStockModal.tsx`	Shipment intake modal supporting bulk packaging selection and live unit piece conversion preview. **Deliverable:** Receive modal calculates unit quantities and updates stock.
#75	VS-16-02	MUST	BACKEND / HANDLERS	**ReceiveStockHandler (owner): multi-line, atomic, optional packaging per line, cost per unit or per container → updates product costSantim** `backend/apps/ledger/handlers/receive_stock.py`	Owner shipment intake command: updates stock balances, appends positive ledger movements, and updates product costSantim. **Deliverable:** ReceiveStockHandler passes multi-line atomic shipment tests.
#76	VS-16-03	MUST	FRONTEND / UI	**Web: Receive Stock modal (segmented unit selector pc | Crate (12), live preview, optional cost per container)** `web/src/features/stock/ReceiveStockForm.tsx`	Segmented unit selector allowing cashiers to record deliveries by piece or crate with live cost calculation. **Deliverable:** Form converts carton wholesale cost into base unit cost.
#77	VS-16-04	MUST	SHARED DOMAIN	**Stock log entries carry packaging snapshot (label, count, baseQtyMilli) for history render** `backend/apps/ledger/models.py`, `web/src/data/schema/stock.ts`	Movement schema preserving packaging metadata (e.g. 'Received 3 crates') for historical audit display. **Deliverable:** Movement entries preserve packaging snapshots in ledger history.
#78	VS-16-05	MUST	FRONTEND / MATH	**Web: Receive cost per container → converts to cost per base unit with half-up rounding** `web/src/core/domain/container-cost.ts`	Client math utility dividing wholesale container cost into base unit santim with half-up rounding. **Deliverable:** Container cost calculations pass vector tests.

VS-17: Manual Stock Adjustments & Shrinkage

Scope: Mandatory reason codes (Damage, Loss, Count Correction), delta adjustments, low-stock threshold badges, and summary cards. | **Count:** 6 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#79	VS-17-01	MUST	BACKEND / HANDLERS	**AdjustStockHandler (owner): ADJUSTMENT entry with reason enum (DAMAGE, LOSS, CORRECTION, OTHER); online rejects negative, offline quarantines** backend/apps/ledger/handlers/adjust_stock.py	Manual stock adjustment service enforcing structured reason codes and mandatory notes for audit compliance. **Deliverable:** AdjustStockHandler passes reason code validation tests.
#80	VS-17-02	MUST	FRONTEND / UI	**Web: Adjust Stock modal (reason picker, quantity input by unit, preview)** web/src/features/stock/AdjustStockModal.tsx	Fast stock adjustment dialog requiring reason selection (Damage, Loss, Count Correction) before saving. **Deliverable:** Modal form enforces mandatory reason code selection.
#81	VS-17-03	MUST	FRONTEND / FORM	**Web: Adjust reason enum with mandatory note for OTHER** web/src/features/stock/adjust-schema.ts	Form validation rule requiring explanatory text when the OTHER adjustment reason is selected. **Deliverable:** Zod schema rejects OTHER reason without accompanying note.
#82	VS-17-04	MUST	FRONTEND / COMPONENT	**Web: Stock screen quick actions bar (Receive/Adjust/Export)** web/src/features/stock/StockActionsBar.tsx	Action bar at the top of the inventory screen providing 1-tap shortcuts to receive stock, adjust stock, or export ledger. **Deliverable:** Actions bar matches stitch prototype layout.
#83	VS-17-05	MUST	FRONTEND / COMPONENT	**Web: Low stock threshold display on product cards** web/src/features/stock/LowStockBadge.tsx	Warning badge tag displayed on product cards when stock drops below configured low-stock threshold. **Deliverable:** Badge displays amber warning when current stock <= threshold.
#84	VS-17-06	MUST	FRONTEND / COMPONENT	**Web: Stock summary cards (Total SKUs, Low Stock Count)** web/src/features/stock/StockSummaryCards.tsx	Summary metric cards displaying active SKU count, total items, and low-stock alert count. **Deliverable:** Summary cards render accurately from local Dexie data.

VS-18: Stock Movement History & Audit Drawer

Scope: Stock overview screen, movement history list, and slide-out audit drawer displaying cryptographic hashes. | **Count:** 2 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#85	VS-18-01	MUST	FRONTEND / UI	**Web: Stock screen card list (mobile) + DataTable (desktop) with Receive/Adjust/History actions** web/src/features/stock/StockScreen.tsx	Central inventory management view displaying stock balances, low-stock alerts, and action triggers. **Deliverable:** Stock screen verified matching stitch prototype layout and mobile responsiveness.
#86	VS-18-02	MUST	FRONTEND / UI	**Web: Stock History drawer/sheet (chronological movements with actor, device, resulting balance)** web/src/features/stock/StockHistoryDrawer.tsx	Slide-out movement audit drawer showing chronological SKU event trail, timestamps, and cryptographic hashes. **Deliverable:** Audit drawer renders immutable event history.

VS-19: Offline Sync Server Push Pipeline & Idempotency

Scope: POST /v1/sync/push, per-shop row locks, ProcessedOperation idempotency, ChangeLog sequence allocation, and outcome matrix. | **Count:** 8 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#87	VS-19-01	MUST	BACKEND / SYNC	**PushService: per-op transaction, shop lock → product locks (ascending ID), re-check ProcessedOperation** backend/apps/sync/services/push.py	Server push engine acquiring shop lock followed by product locks in deterministic ascending ID order to prevent deadlocks. **Deliverable:** PushService passes deadlock stress tests under concurrent multi-device push.
#88	VS-19-02	MUST	BACKEND / SYNC	**Idempotency: ProcessedOperation table (opId, payloadHash, outcome, resultJson); same hash = stored result; different hash = IDEMPOTENCY_KEY_REUSE** backend/apps/sync/models/processed_op.py	PostgreSQL idempotency tracking table caching operation results. Replays stored result on duplicate opId; rejects key reuse on payload mismatch. **Deliverable:** Idempotency tests verify duplicate opId returns identical response (AT-04).
#89	VS-19-03	MUST	BACKEND / SYNC	**OperationDispatcher registry (built in config/container.py): maps OperationType → (payload parser, handler); architecture test asserts completeness** backend/apps/sync/dispatcher.py	Type-safe operation registry routing incoming sync operations (RECORD_SALE, RECEIVE_STOCK, ADJUST_STOCK) to appropriate domain handlers. **Deliverable:** Architecture test asserts 100% operation type coverage in dispatcher.
#90	VS-19-04	MUST	BACKEND / SYNC	**dependsOn resolution: dependent op gets REJECTED_DEPENDENCY or quarantined into same case** backend/apps/sync/dependency_resolver.py	Batch dependency resolver ensuring operations referencing prior operations within the same batch execute in causal order. **Deliverable:** Dependency resolver handles dependent operation failures gracefully.
#91	VS-19-05	MUST	BACKEND / SECURITY	**Lease validation at receipt time: membership ACTIVE, lease valid, device matches, op type allowed, clientCreatedAt in window ±5min** backend/apps/sync/lease_validator.py	Server sync gate verifying device lease validity, temporal sync window, and operation permissions before execution. **Deliverable:** Lease validation tests verify expired leases are rejected.
#92	VS-19-06	MUST	BACKEND / SYNC	**Outcome matrix implementation (§6.8 table) for all operation types × situations** backend/apps/sync/outcome_matrix.py	Implementation of SDD §6.8 outcome matrix covering all operation types across active, oversold, and deactivated device states. **Deliverable:** Comprehensive test suite covering all outcome matrix scenarios.
#93	VS-19-07	MUST	BACKEND / SYNC	**ChangeLog allocation under shop lock: shop.syncSeq += 1 per touched entity; written to ChangeLog + StockChange.serverSeq + Product.updatedSeq** backend/apps/sync/change_log.py	Monotonic sequence number generator allocating gap-free sequence IDs per shop under the shop row lock. **Deliverable:** ChangeLog sequence tests assert strictly increasing gap-free numbering.
#94	VS-19-08	MUST	BACKEND / API	**Push response: per-op OperationResult with entries, stockAfter, conflictCaseId, reasonCode, replayed, noop** backend/apps/sync/serializers.py	Structured push response contract detailing exact outcome, confirmed server sequence, and resulting stock balance for each operation. **Deliverable:** Push response schema validated against OpenAPI contract.

VS-20: Client Outbox Dispatcher & Multi-Tab Web Lock

Scope: Client outbox batch push (<=50 ops), exponential backoff with jitter, Web Lock coordinator, and ResultApplier. | **Count:** 4 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#95	VS-20-01	MUST	FRONTEND / SYNC	**Web push stage: batch ≤50 ops, ≤200KB, ascending deviceSeq; retry same opId on transport failure; poison guard (8 failures → ATTENTION_REQUIRED)** web/src/sync/push-stage.ts	Client outbox dispatcher packaging batches of up to 50 operations in deviceSeq order. Poison pill guard flags unprocessable operations. **Deliverable:** Push stage tests verify batch transmission and poison pill quarantine.
#96	VS-20-02	MUST	FRONTEND / SYNC	**Web backoff: exponential (base 2s, factor 2, cap 5min, full jitter)** web/src/sync/backoff.ts	Network retry algorithm using exponential backoff with full jitter to avoid thundering herd server overload when connectivity returns. **Deliverable:** Backoff tests verify delay bounds and randomized jitter.
#97	VS-20-03	MUST	FRONTEND / SYNC	**Web: Sync Engine single-flight Web Lock (stokari-sync-{shopId}) for multi-tab safety** web/src/sync/web-lock.ts	Single-leader election using Web Locks API (navigator.locks.request) ensuring only one browser tab syncs at a time. **Deliverable:** Multi-tab simulation test verifies only one tab acquires sync lock.
#98	VS-20-04	MUST	FRONTEND / SYNC	**Web: Result Applier — updates Dexie rows in one atomic transaction (§5.10)** web/src/sync/result-applier.ts	Client sync consumer updating local Dexie records with confirmed server IDs, clearing outbox rows, and updating projection balances. **Deliverable:** Result applier passes atomic Dexie update tests.

VS-21: Offline Sync Pull Pipeline & Device Hydration

Scope: GET /v1/sync/pull cursor-based deltas, initial bootstrap, CURSOR_RESET, reachability probe, and header SyncPill. | **Count:** 8 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#99	VS-21-01	MUST	BACKEND / SYNC	**PullService: GET /v1/sync/pull? cursor=N&limit=200 → next limit ChangeLog rows → load current entities (role-filtered) → return nextCursor, hasMore, headSeq, serverTime** backend/apps/sync/services/pull.py	Cursor-based delta synchronization service loading modified entities since cursor N, filtered by caller role. **Deliverable:** PullService integration tests verify delta pagination and headSeq tracking.
#100	VS-21-02	MUST	BACKEND / API	**Pull response: products, stockChanges, stockLevels, sales, saleCancellations, actors, conflicts (owner only)** backend/apps/sync/views.py	Complete pull DTO contract returning delta arrays for products, ledger movements, sales, and conflicts. **Deliverable:** Pull endpoint verified with role-based field stripping.
#101	VS-21-03	MUST	FRONTEND / SYNC	**Web pull stage: ONE Dexie tx — upsert by id, apply stockLevels only if version > confirmedVersion, recompute pendingDelta, update pullCursor** web/src/sync/pull-stage.ts	Client pull consumer updating local IndexedDB cache in a single atomic transaction without overwriting pending local operations. **Deliverable:** Pull stage integration test passes asserting cursor advancement.
#102	VS-21-04	MUST	FRONTEND / SYNC	**New-device bootstrap: cursor=0 → pull pages until hasMore=false → integrity check → ready** web/src/sync/bootstrap.ts	Initial device hydration loading entire store catalog and confirmed stock levels onto a clean device. **Deliverable:** Bootstrap routine hydrates 2,000 products cleanly.
#103	VS-21-05	MUST	FRONTEND / SYNC	**CURSOR_RESET handling: restart from 0 without touching outbox** web/src/sync/cursor-reset.ts	Recovery protocol recovering from expired server cursors by restarting pull from 0 without touching pending outbox records. **Deliverable:** Cursor reset test asserts pending outbox operations preserved.
#104	VS-21-06	MUST	BACKEND / HEALTH	**Health probe: GET /health (2s timeout, no DB heavy work); used for online/offline decision, warm-up** backend/backend/views.py	Lightweight health probe returning server timestamp for client clock-skew detection. **Deliverable:** Health probe returns 200 OK in < 10ms.
#105	VS-21-07	MUST	FRONTEND / SYNC	**Web: Sync trigger schedule (app open, online event, tab visible, debounced 500ms, timer 30s)** web/src/sync/sync-coordinator.ts	Event-driven sync coordinator triggering on app launch, network online event, tab visibility, and periodic 30s timer. **Deliverable:** Coordinator triggers sync on network reconnect.
#106	VS-21-08	MUST	FRONTEND / UI	**Web: SyncPill in header with popover (last sync, outbox count, storage, persistent status, 'Sync Now')** web/src/ui/SyncPill.tsx	Interactive header badge displaying 4 sync states (Synced, Syncing, Offline, Conflict) with detail popover. **Deliverable:** SyncPill verified matching stitch prototype styling and interactive popover.

VS-22: AT-06 Conflict Detection & Review Queue

Scope: StockBecameNegative event, ConflictCase model, Sell Anyway warning sheet, and owner review queue with parallel timeline. | **Count:** 8 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#107	VS-22-01	MUST	BACKEND / EVENTS	**Domain event: StockBecameNegative published in-transaction; ConflictService subscribes → opens/extends OFFLINE_OVERSELL case, sets underReview** backend/apps/ledger/events.py, backend/apps/conflicts/subscribers.py	In-transaction domain event trigger: when balance drops below zero, flags StockLevel.under_review = True and opens an AT-06 ConflictCase. **Deliverable:** Integration test asserting oversell opens conflict case in same transaction (AT-06).
#108	VS-22-02	MUST	FRONTEND / UI	**Web: Sell Anyway Alert Sheet (warns confirmed vs projected, flags for review, destructive button)** web/src/features/pos/SellAnywaySheet.tsx	Explicit confirmation sheet warning cashier that sale exceeds stock, allowing them to proceed while tagging sale for owner review (AT-06). **Deliverable:** Sell Anyway sheet records sale with needs_review flag.
#109	VS-22-03	MUST	BACKEND / MODELS	**ConflictCase model: type, productId, status, version, stateSnapshot, items[], quarantined[], resolution, resolvedBy** backend/apps/conflicts/models.py	ConflictCase table with optimistic locking version, conflicting sale line references, and audit resolution snapshots. **Deliverable:** Conflict models migration running cleanly on PostgreSQL.
#110	VS-22-04	MUST	BACKEND / CONFLICTS	**ConflictService: opens/extends cases on StockBecameNegative, PRODUCT_INACTIVE, CANCELLATION_CONFLICT, INVALID_STOCK_OPERATION** backend/apps/conflicts/services.py	Conflict engine grouping concurrent oversell events on the same product into a single open conflict case. **Deliverable:** ConflictService groups multiple oversell events into single case (AT-06).
#111	VS-22-05	MUST	FRONTEND / UI	**Web: Review queue (owner) — card list (mobile) + split-pane (desktop) with case list + detail drawer** web/src/features/conflicts/ConflictReviewScreen.tsx	Owner arbitration dashboard displaying active conflict queue with parallel cashier timelines. **Deliverable:** Conflict review screen matches stitch prototype layout and mobile responsiveness.
#112	VS-22-06	MUST	FRONTEND / COMPONENT	**Web: Conflict case card — type badge, product, current balance, timeline preview (last 3 events)** web/src/features/conflicts/ConflictCaseCard.tsx	Card component displaying product, negative balance amount, and conflicting cashiers. **Deliverable:** Conflict card renders with appropriate severity badges.
#113	VS-22-07	MUST	FRONTEND / UI	**Web: Conflict case detail drawer — discrepancy alert, concurrent timeline, 3 resolution actions** web/src/features/conflicts/ConflictDetailDrawer.tsx	Parallel timeline comparison view showing Register 1 Hana vs Register 2 Dawit concurrent sales with deficit summary. **Deliverable:** Detail drawer renders concurrent timeline and resolution buttons.
#114	VS-22-08	MUST	FRONTEND / POLISH	**Mobile Sell Anyway Sheet (destructive variant, confirmed vs projected)** web/src/features/pos/SellAnywayMobile.tsx	High-contrast warning dialog approving offline overselling on mobile screens. **Deliverable:** Alert dialog conforms to WCAG 2.2 touch-target and contrast rules.

VS-23: Conflict Arbitration & Owner Resolution Actions

Scope: ResolveConflictHandler with optimistic locking, 3 resolution actions (Restock, Cancel, Acknowledge), and in-app notifications. | **Count:** 7 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#115	VS-23-01	MUST	BACKEND / HANDLERS	**ResolveConflictHandler (owner): validates caseVersion, applies RECONCILE_STOCK / CANCEL_SALE / ACKNOWLEDGE via StockLogService, closes case, clears underReview** backend/apps/conflicts/handlers/resolve.py	Owner resolution service executing corrective ledger movements, closing the conflict case, and clearing product under_review flag. **Deliverable:** ResolveConflictHandler closes case and clears underReview (AT-06).
#116	VS-23-02	MUST	BACKEND / CONFLICTS	**Stale resolution: caseVersion mismatch → NEEDS_REVIEW + STALE_RESOLUTION event, case stays OPEN** backend/apps/conflicts/handlers/stale_resolution.py	Optimistic concurrency guard preventing conflicting owner resolutions if new concurrent sales occurred while reviewing. **Deliverable:** Stale resolution test asserts case remains OPEN on version conflict.
#117	VS-23-03	MUST	FRONTEND / UI	**Web: Resolution actions — Record Restock (qty + note), Cancel Sale (select + reason), Acknowledge Deficit** web/src/features/conflicts/ResolutionActionModals.tsx	3 functional owner resolution action dialogs executing atomic corrective actions. **Deliverable:** Action modals execute respective conflict resolution endpoints.
#118	VS-23-04	MUST	SHARED DOMAIN	**Notifications: CASE_OPENED, CASE_EXTENDED, QUARANTINE_PENDING → owner in-app + GET /notifications** backend/apps/conflicts/notifications.py, web/src/features/notifications/NotificationDrawer.tsx	In-app notification system alerting owner to overselling events and quarantine items. **Deliverable:** Notification drawer renders unread conflict alerts.
#119	VS-23-05	MUST	FRONTEND / UI	**Web: Notifications screen (list + mark read)** web/src/features/notifications/NotificationScreen.tsx	Notification view listing store alerts with 1-tap mark read actions. **Deliverable:** Notification screen marks items read and updates badge.
#120	VS-23-06	MUST	FRONTEND / STATE	**Web: Conflict state machine: OPEN → RESOLVED only via explicit ResolveConflict** web/src/features/conflicts/state-machine.ts	State machine preventing automatic or accidental dismissal of conflict cases. **Deliverable:** State machine asserts cases stay OPEN until explicit resolution.
#121	VS-23-07	MUST	BACKEND / TEST	**Architecture test: every ConflictType has resolution paths; no auto-close** backend/tests/architecture/test_conflict_completeness.py	Automated architectural test verifying zero unhandled conflict states in backend. **Deliverable:** Architecture test asserts completeness of resolution paths.

VS-24: Deactivated Staff Drain Flow & Security Quarantine

Scope: 10-minute drain token JWT, quarantine isolation (zero ledger impact), and owner Accept/Reject inspection tab. | **Count:** 5 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#122	VS-24-01	MUST	BACKEND / SYNC	**Quarantine path: stores full payload, QUARANTINED outcome, AUTHORIZATION_REVIEW case, no stock log effect** backend/apps/sync/quarantine.py	Quarantine storage isolating unauthorized or suspicious operations without applying any mutations to the stock ledger. **Deliverable:** Quarantine tests assert quarantined operation produces zero ledger entries.
#123	VS-24-02	MUST	BACKEND / HANDLERS	**ReviewQuarantineHandler (owner): ACCEPT_QUARANTINED (re-runs original handler as original actor, returns inner[] per-op results) / REJECT_QUARANTINED** backend/apps/conflicts/handlers/quarantine.py	Quarantine review service executing late operations retrospectively or permanently rejecting them with an audit reason. **Deliverable:** Quarantine review handler executes accepted operations.
#124	VS-24-03	MUST	FRONTEND / UI	**Web: Quarantined operations tab — table with Accept/Reject per operation or bulk per device** web/src/features/conflicts/QuarantinedOperationsTab.tsx	Inspection table displaying operations sent by deactivated cashier devices with Accept/Reject actions. **Deliverable:** Quarantined tab renders with audit reason inputs.
#125	VS-24-04	MUST	FRONTEND / USECASE	**Web: Quarantine Accept → re-runs handler, shows per-op results in inner[]** web/src/features/conflicts/usecases/acceptQuarantine.ts	Client action triggering server re-execution of quarantined operation. **Deliverable:** Accept quarantine updates local state with confirmed sale.
#126	VS-24-05	MUST	FRONTEND / USECASE	**Web: Quarantine Reject → archives with reason, never deletes** web/src/features/conflicts/usecases/rejectQuarantine.ts	Client action archiving quarantined operation with permanent audit note. **Deliverable:** Reject quarantine archives record without deleting.

VS-25: Today's Revenue Dashboard (3-Bucket Accounting)

Scope: Offline Dexie aggregation grouping revenue into 3 mutually exclusive buckets (Confirmed, Pending, Review), and Today screen. | **Count:** 6 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#127	VS-25-01	MUST	FRONTEND / DASHBOARD	**Offline dashboard (Web): three buckets — Confirmed (SYNCED), Local Pending (WAITING_TO_SYNC/SYNCING), Needs Review (NEEDS_REVIEW + quarantined) — no double counting** web/src/features/dashboard/useTodayRevenue.ts	Client-side aggregation engine reading Dexie sales tables and grouping revenue into 3 mutually exclusive buckets (Confirmed, Pending, Review). **Deliverable:** Revenue buckets calculate with zero double-counting (AT-13).
#128	VS-25-02	MUST	BACKEND / REPORTS	**Server dashboard endpoint: GET /reports/dashboard?date= → DashboardReport (confirmed sales, needs-review sales, by payment label, low stock, open cases, quarantined count)** backend/apps/reporting/dashboard.py	Server SQL aggregation query returning daily financial summary, payment label breakdowns, and open conflict counts. **Deliverable:** Dashboard endpoint returns confirmed revenue and review counts.
#129	VS-25-03	MUST	FRONTEND / COMPONENT	**Web: RevenueBucket component — 3 cards (Confirmed=Emerald, Pending=Amber, Review=Crimson) with amount + count** web/src/features/dashboard/RevenueBucketCards.tsx	Tactile dashboard cards with high-contrast status colors (Emerald, Amber, Crimson) displaying ETB totals and transaction counts. **Deliverable:** Revenue cards match stitch prototype layout and responsive scaling.
#130	VS-25-04	MUST	FRONTEND / UI	**Web: Today screen — date selector (EAT), 3 buckets, valuation (owner), low stock list, quick actions** web/src/features/dashboard/TodayDashboardScreen.tsx	Central store management dashboard displaying today's financial metrics, inventory valuation, and stock alerts. **Deliverable:** Today screen renders cleanly across mobile (390px) and desktop (1280px).
#131	VS-25-05	MUST	FRONTEND / UI	**Web: Dashboard density toggle (compact/comfortable table rows)** web/src/features/dashboard/DensityToggle.tsx	View density switcher allowing users to toggle between compact and comfortable data row spacing. **Deliverable:** Density toggle updates UI layout spacing instantaneously.
#132	VS-25-06	MUST	SHARED DOMAIN	**Date handling: Africa/Addis_Ababa timezone, localDay on sales for grouping** web/src/core/time/eat-date.ts, backend/common/time.py	Timezone normalizer mapping UTC timestamps to East Africa Time (UTC+3) calendar dates for sales grouping. **Deliverable:** Date grouping functions pass daylight and boundary tests.

VS-26: Inventory Capital Valuation & Restock Alerts

Scope: Store asset valuation calculation (sum max(qty, 0) x costSantim), uncosted item helper text, and restock warning cards. | **Count:** 3 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#133	VS-26-01	MUST	BACKEND / REPORTS	**Inventory valuation (owner): GET /reports/inventory-value → Σ max(qty,0) × costSantim over products with cost; approximate flag if any negative/under-review/uncosted** backend/apps/reporting/valuation.py	SQL query computing total inventory asset value, tagging approximate if any products have negative stock or missing wholesale costs. **Deliverable:** Valuation endpoint returns capital sum and uncosted SKU counts.
#134	VS-26-02	MUST	FRONTEND / COMPONENT	**Web: ValuationCard — large total, helper line 'N items have no cost recorded; M items approximate'** web/src/features/dashboard/ValuationCard.tsx	Capital valuation widget displaying total store asset value in ETB with uncosted item helper text. **Deliverable:** ValuationCard renders in today_dashboard view.
#135	VS-26-03	MUST	FRONTEND / COMPONENT	**Web: Low stock warning cards — product name, current/threshold with unit, Restock button** web/src/features/dashboard/LowStockAlertCard.tsx	Alert cards highlighting products below reorder thresholds with 1-tap shortcut to receive shipment. **Deliverable:** Low stock alert links directly to receive stock modal.

VS-27: Data Backup & Emergency Outbox Export

Scope: Streaming CSV exports, client emergency backup file generator (stokari-pending-export/v1.json), and export buttons. | **Count:** 3 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#136	VS-27-01	SHOULD	BACKEND / EXPORT	**CSV exports [Should]: GET /exports/sales.csv, /stock-changes.csv (owner only)** backend/apps/reporting/csv_exports.py	Streaming CSV export endpoints for tax accounting, bookkeeping, and external ledger reconciliation. **Deliverable:** CSV exports stream large datasets without memory exhaustion.
#137	VS-27-02	MUST	FRONTEND / EXPORT	**Pending-records export (device): JSON stokari-pending-export/v1 from outbox/sales/stockChanges — never marked confirmed, sensitive data warning** web/src/data/export-pending.ts	Emergency client backup utility generating an unencrypted JSON export of all unsynced local outbox records to prevent data loss if phone is damaged. **Deliverable:** Export function generates valid stokari-pending-export/v1 JSON.
#138	VS-27-03	MUST	FRONTEND / UI	**Web: Export Pending Records button in Settings + Staff screens** web/src/features/settings/ExportBackupButton.tsx	Functional export button triggering actual client-side file download with sensitive data warning dialog. **Deliverable:** Button triggers browser download of backup JSON file.

VS-28: Physical Inventory Count & Variance Reconciliation

Scope: CountSession model, shelf count entry sheet, server proposed variance accounting for intermediate sales, and COUNT_CORRECTION movement. | **Count:** 6 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#139	VS-28-01	SHOULD	BACKEND / MODELS	**CountSession + CountLine models [Should]** backend/apps/counts/models.py	Physical inventory count schema tracking observed shelf counts against recorded ledger balances. **Deliverable:** Count models migration running cleanly on PostgreSQL.
#140	VS-28-02	SHOULD	BACKEND / HANDLERS	**SubmitCountHandler, DecideCountHandler [Should]** backend/apps/counts/handlers.py	Command handlers to record observed physical counts and apply owner variance decisions. **Deliverable:** Count handlers pass variance calculation tests.
#141	VS-28-03	SHOULD	BACKEND / QUERIES	**Server proposes delta = observedMilli − balanceAtVersion(baseVersion) (stock log query); shows movements after baseVersion [Should]** backend/apps/counts/reconciliation.py	Reconciliation query calculating stock shrinkage delta while accounting for intermediate sales. **Deliverable:** Proposed delta correctly accounts for sales made during counting.
#142	VS-28-04	SHOULD	BACKEND / HANDLERS	**Owner edits delta, approves → COUNT_CORRECTION entry [Should]** backend/apps/counts/approval.py	Owner approval workflow appending COUNT_CORRECTION movements to the stock ledger. **Deliverable:** Approval emits audit adjustment ledger entry.
#143	VS-28-05	SHOULD	FRONTEND / UI	**Web: Count screen — start session, enter observed per product, review proposed delta, decide [Should]** web/src/features/counts/PhysicalCountScreen.tsx	Interactive shelf count counting view with live variance feedback. **Deliverable:** Count screen renders product input rows and delta previews.
#144	VS-28-06	SHOULD	FRONTEND / UI	**Web: Count session history [Should]** web/src/features/counts/CountHistoryScreen.tsx	Historical log of past physical counts and shrinkage adjustments. **Deliverable:** Count history renders past reconciliation sessions.

VS-29: Standalone Local-Only Kiosk Mode

Scope: meta.mode = 'LOCAL_ONLY' toggle, persistent offline banner, export/import migration bundle, and transition documentation. | **Count:** 4 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#145	VS-29-01	SHOULD	FRONTEND / LOCAL	**meta.mode = 'LOCAL_ONLY': no tokens, no outbox transmission, no pull; persistent banner [Should]** web/src/data/local-mode.ts	Standalone mode operating purely offline without server authentication or cloud sync. **Deliverable:** Local-only mode disables outbox transmission and shows banner.
#146	VS-29-02	SHOULD	DOCUMENTATION	**Explicit migration procedure to shared mode (out of MVP scope) [Should]** docs/LOCAL_TO_CLOUD_MIGRATION.md	Documented procedure for migrating a standalone local-only shop into the cloud platform. **Deliverable:** Migration guide reviewed and committed.
#147	VS-29-03	SHOULD	FRONTEND / MIGRATION	**Data export/import for local-only → shared migration [Should]** web/src/data/migration-bundle.ts	JSON bundle exporter and importer for transitioning standalone local shops to cloud tenancy. **Deliverable:** Migration bundle passes import/export roundtrip tests.
#148	VS-29-04	SHOULD	FRONTEND / UI	**Web: Local-only mode toggle in Settings with warning [Should]** web/src/features/settings/LocalOnlyToggle.tsx	Settings toggle allowing a shop to switch into standalone unclouded operation. **Deliverable:** Toggle displays confirmation warning before enabling.

VS-30: Design System Primitives & Responsive Shell

Scope: CSS variables (Cobalt #1E40AF), primitive component library with 44px targets, desktop sidebar, virtualized DataTable, and light-only audit. | **Count:** 8 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#149	VS-30-01	MUST	FRONTEND / TOKENS	**Design token implementation: CSS variables for all colors, spacing, radii, shadows, typography** web/src/index.css	CSS variables implementing deep cobalt (#1E40AF), neutral slate canvas, and hairline borders. **Deliverable:** Design tokens used consistently across components.
#150	VS-30-02	MUST	FRONTEND / COMPONENTS	**Component library: Button, Input, Badge, Card, Modal, Sheet, Drawer, DataTable with all variants** web/src/ui/*.tsx	Accessible primitive component library with 44px minimum touch targets and clear focus rings. **Deliverable:** Component library passes unit and accessibility tests.
#151	VS-30-03	MUST	FRONTEND / COMPONENTS	**Specialized components: SyncPill, QuantityInput, CurrencyInput, UnitSelect, StockLevelDisplay, PaymentSelector** web/src/ui/specialized/*.tsx	Domain-specific composite components for retail transactions and stock display. **Deliverable:** Specialized components tested with mock data.
#152	VS-30-04	MUST	FRONTEND / COMPONENTS	**Composite components: RevenueBucket, ConflictCaseCard, LeaseHealthRow, ProductCard, LowStockCard** web/src/ui/composite/*.tsx	Dashboard and conflict domain card components rendering complex metrics cleanly. **Deliverable:** Composite components render across all responsive breakpoints.
#153	VS-30-05	MUST	FRONTEND / LAYOUT	**Desktop sidebar (240px → 48px icon rail), sticky header, split-pane drawers** web/src/app/DesktopShell.tsx	Responsive layout shell expanding into desktop sidebar on >1024px displays with icon rail collapsing. **Deliverable:** Desktop layout scales cleanly without content distortion.
#154	VS-30-06	MUST	FRONTEND / COMPONENT	**DataTable: sticky header, sortable columns, filter toolbar, bulk actions, density toggle** web/src/ui/DataTable.tsx	Virtualizable data table component with sticky header and horizontal scroll wrapper for data density. **Deliverable:** DataTable handles 2,000 rows with 60fps scrolling.
#155	VS-30-07	MUST	FRONTEND / A11Y	**Reduced motion: pulse animations disabled, slide transitions instant** web/src/index.css	prefers-reduced-motion CSS media queries disabling pulse animations and slide transitions. **Deliverable:** Animations disabled when reduced-motion is requested.
#156	VS-30-08	MUST	FRONTEND / AUDIT	**Dark mode: NOT SUPPORTED — verify light-only implementation** docs/DESIGN.md	Verification audit confirming pure high-contrast light-only daytime shop theme per task 188 specification. **Deliverable:** Audit confirms light-mode only consistency across all screens.

VS-31: Concurrency, Property & End-to-End Simulation

Scope: Multi-threaded race condition tests, Hypothesis property tests (1,000 runs), fast-check, Playwright two-device conflict simulation (AT-06). | **Count:** 9 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#157	VS-31-01	MUST	BACKEND / TEST	**API unit tests: money, quantity, status machines, permission table, conflict policy, projection** backend/tests/unit/*.py	Backend domain unit test suite achieving >90% coverage on core business logic. **Deliverable:** Unit test suite passes in < 5 seconds.
#158	VS-31-02	MUST	BACKEND / TEST	**API integration tests (pytest-django + factory_boy + Postgres): every handler, repository, endpoint** backend/tests/integration/*.py	PostgreSQL integration tests exercising every command handler, repository, and DRF endpoint. **Deliverable:** Integration test suite passes on local PostgreSQL database.
#159	VS-31-03	MUST	BACKEND / CONCURRENCY	**API concurrency tests: parallel pushes, duplicate opId, pull during push (transaction=True + threads)** backend/tests/concurrency/test_sync_concurrency.py	Multi-threaded race condition tests proving shop-level row lock prevents lost updates and deadlocks. **Deliverable:** Concurrency tests pass with 10 simulated parallel devices.
#160	VS-31-04	MUST	BACKEND / PROPERTY	**API property tests (Hypothesis): random interleavings → qty = Σ deltas, no duplicate effect, cursor safety** backend/tests/property/test_ledger_properties.py	Hypothesis property tests asserting stock ledger mathematical invariants across 1,000 randomized operation sequences. **Deliverable:** Property tests run 1,000 randomized iterations without invariant failure.
#161	VS-31-05	MUST	BACKEND / ARCHITECTURE	**API architecture tests: one handler per op, no direct Model.objects, composite keys present, no business logic in models** backend/tests/architecture/test_layer_boundaries.py	Import-linter rules asserting clean architectural boundaries and zero direct ORM calls from views. **Deliverable:** Architecture test suite passes.
#162	VS-31-06	MUST	FRONTEND / TEST	**Web unit tests (Vitest + fake-indexeddb): use cases, outbox, result applier, recovery, projection** web/src/**/__tests__/*.test.ts	Frontend unit test suite achieving >90% coverage using fake-indexeddb. **Deliverable:** Vitest suite passes with zero warnings.
#163	VS-31-07	MUST	FRONTEND / PROPERTY	**Web property tests (fast-check): projection and outbox state** web/src/data/__tests__/projection-property.test.ts	Fast-check property tests verifying client projection mathematical invariants across randomized outbox states. **Deliverable:** Fast-check property tests pass 500 iterations.
#164	VS-31-08	MUST	E2E / CONCURRENCY	**E2E tests (Playwright): two browser contexts as two devices; offline sale → sync → reload → conflict flow** web/e2e/two-device-conflict.spec.ts	Automated two-device conflict simulation verifying AT-06: Device 1 sells 7, Device 2 sells 6 from stock 10, asserting ledger -3 and case opened. **Deliverable:** Two-device Playwright test passes cleanly.
#165	VS-31-09	MUST	QA / DEVICE	**Device tests: real low-end Android (2GB RAM), 320px emulator — install, offline open, NFR-01/02/14** docs/DEVICE_VERIFICATION_REPORT.md	Low-spec hardware verification verifying PWA launch in < 3s and smooth scrolling on 320px viewport. **Deliverable:** Device verification report documented and approved.

VS-32: Quality Gates, Accessibility & Traceability Enforcement

Scope: trace_check.py requirement gate, WCAG 2.1 AA axe-core audit, bundle size <=2MB, usability study with 5 users, and visual regressions. | **Count:** 13 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#166	VS-32-01	MUST	QA / PERF	**Performance: Lighthouse + timers on target phone (2000 products, 50k movements); p95 ≤300ms local actions, search <1s, launch <3s, dashboard <2s** docs/PERFORMANCE_BENCHMARK_REPORT.md	Performance benchmark on simulated low-spec Android device meeting all NFR latency requirements. **Deliverable:** All NFR performance benchmarks verified and documented.
#167	VS-32-02	MUST	FRONTEND / BUILD	**Bundle size: ≤2MB compressed (code splitting, lazy routes)** web/vite.config.ts	Production bundle optimization using route-level code splitting and tree shaking. **Deliverable:** Compressed production bundle verified <= 2MB.
#168	VS-32-03	MUST	QA / A11Y	**Accessibility: WCAG 2.1 AA, 44px touch targets, 320px no horizontal scroll, status = text + icon** docs/ACCESSIBILITY_AUDIT_REPORT.md	Full accessibility review asserting 44px touch targets and non-color dependent status indicators. **Deliverable:** Audit report confirms WCAG 2.1 AA compliance.
#169	VS-32-04	MUST	QA / UX	**Usability test: ≥5 outside users, ≥80% complete basic sale on first attempt** docs/USABILITY_TEST_REPORT.md	Usability study with external users validating cashier checkout completion in under 10 seconds. **Deliverable:** Usability test report documenting >= 80% first-attempt success.
#170	VS-32-05	MUST	FRONTEND / UI	**Web: Clock skew banner (>5 min) with server time offset** web/src/ui/ClockSkewBanner.tsx	Warning banner displayed at the top of the app if client device clock differs from server time by more than 5 minutes. **Deliverable:** Banner displays with computed offset when time drift is detected.
#171	VS-32-06	MUST	FRONTEND / UI	**Web: Error toasts with retry action (no bare 'something went wrong')** web/src/ui/ToastManager.tsx	Context-sensitive toast notification system with machine-readable error codes and 1-tap retry actions. **Deliverable:** Toasts render with appropriate semantic color and retry callbacks.
#172	VS-32-07	MUST	FRONTEND / UI	**Web: Empty states with CTA button (no illustrations)** web/src/ui/EmptyState.tsx	Utilitarian empty state component guiding users to action (e.g., 'No staff members yet - Invite Attendant'). **Deliverable:** Clean empty state renders without heavy decorative SVGs.
#173	VS-32-08	MUST	CI / QUALITY	**Traceability: @pytest.mark.req('FR-14', 'AT-04') + web test titles [FR-14] [AT-04]; trace_check.py fails CI if Must req untagged** scripts/trace_check.py	Quality gate script verifying every Must requirement in SRS v0.5 is linked to at least one automated test. **Deliverable:** trace_check.py passes with 100% requirement coverage.
#174	VS-32-09	MUST	DOCUMENTATION	**AT-01..AT-20 mapping to test scenarios (T-01..T-35)** docs/ACCEPTANCE_TEST_MATRIX.md	Comprehensive traceability matrix document linking all 20 acceptance tests to automated test cases. **Deliverable:** Acceptance test matrix document committed.
#175	VS-32-10	MUST	CI / QUALITY	**Coverage gates: 70% overall, 90% on domain/, core/, sync/** pyproject.toml, vite.config.ts	Hard coverage thresholds in CI enforcing 90% test coverage on critical business logic. **Deliverable:** Pull requests blocked if coverage drops below threshold.
#176	VS-32-11	MUST	QA / VISUAL	**Visual regression tests: all 4 states × 3 breakpoints × 2 roles per component** web/e2e/visual-regression.spec.ts	Automated snapshot tests asserting visual stability across desktop, tablet, and mobile viewports. **Deliverable:** Visual snapshots committed and passing.
#177	VS-32-12	MUST	QA / A11Y	**Accessibility audit: axe-core in CI, manual screen reader test** web/e2e/accessibility.spec.ts	Axe-core automated accessibility audit enforcing WCAG 2.1 AA compliance. **Deliverable:** Zero critical accessibility violations across all screens.
#178	VS-32-13	MUST	CI / PERF	**Performance regression: bundle size, TTI, sync latency tracked per PR** .github/workflows/perf-tracking.yml	Automated bundle size and latency tracking on every pull request. **Deliverable:** PR comments report bundle size and regression diffs.

VS-33: Deferred Cloud Infrastructure & Disaster Recovery

Scope: Post-development cloud deployment: Render API, Vercel web, Neon cloud database, Sentry, and nightly backup restore tests. | **Count:** 10 tasks

ID	Code	Priority	Layer	Task Title & Target Files	Architectural Scope & Deliverable
#179	VS-33-01	MUST	CLOUD CI/CD	**GitHub Actions API CI: ruff, mypy, import-linter, file-size, pytest (Postgres), check --deploy, OpenAPI diff** `.github/workflows/api-ci.yml`	Automated backend CI workflow running linters, tests, and deployment verification on pull requests. **Deliverable:** Green badge on main/dev branches.
#180	VS-33-02	MUST	CLOUD CI/CD	**GitHub Actions Web CI: lint, types, dependency-cruiser, tests, bundle-size, trace-check** `.github/workflows/web-ci.yml`	Automated frontend CI workflow running linters, tests, and bundle size tracking. **Deliverable:** Green badge on main/dev branches.
#181	VS-33-03	MUST	CLOUD HOSTING	**Vercel (web) + Render (API) deployment pipelines with staging/prod** `render.yaml, vercel.json`	Cloud hosting deployment pipelines with staging and production environment branches. **Deliverable:** Live staging deployment accessible over HTTPS.
#182	VS-33-04	MUST	CLOUD DATABASE	**Neon PostgreSQL dev/staging/prod branches + MIGRATION_DATABASE_URL** `backend/backend/settings.py`	Neon serverless PostgreSQL branching with isolated staging and production connection pooling. **Deliverable:** Automated database branch migrations functional.
#183	VS-33-05	MUST	CLOUD MONITORING	**Sentry + UptimeRobot + nightly pg_dump cron + nightly reconciliation cron** `.github/workflows/nightly-backup.yml`	Production error tracking with Sentry, external uptime pinging, and nightly automated logical dumps. **Deliverable:** Monitoring active and backup cron scheduled.
#184	VS-33-06	MUST	PRODUCTION HARDENING	**Django hardening: DEBUG=False, ALLOWED_HOSTS, no admin/auth/sessions/CSRF, browsable API disabled, SECURE_SSL_REDIRECT, HSTS, CSP** `backend/backend/settings_prod.py`	Production Django security settings disabling unused middleware and enforcing strict HSTS and CSP headers. **Deliverable:** Django deployment check passes with zero warnings (check --deploy).
#185	VS-33-07	MUST	PRODUCTION SECURITY	**Rate limits: push/pull 60 req/min/user, auth 10 req/min/IP (shared cache table across gunicorn workers)** `backend/common/http/throttles.py`	Distributed Redis or PostgreSQL cache-backed rate limiting protecting auth and sync endpoints. **Deliverable:** Rate limits enforced across concurrent requests.
#186	VS-33-08	MUST	PRODUCTION SECURITY	**Body limits: 200KB push, 50 ops max; DATA_UPLOAD_MAX_MEMORY_SIZE hard cap** `backend/backend/settings.py`	Hard memory and upload payload caps preventing memory exhaustion attacks on sync endpoints. **Deliverable:** Uploads > 200KB rejected with 413 Payload Too Large.
#187	VS-33-09	MUST	PRODUCTION LOGGING	**Logging: JSON with requestId, shopId, opId only; no tokens/emails/payloads** `backend/common/logging.py`	Structured JSON logging for Datadog or CloudWatch scrubbed of sensitive PII, passwords, and tokens. **Deliverable:** Logs output clean structured JSON without credentials.
#188	VS-33-10	MUST	DISASTER RECOVERY	**Backup/restore test: staging restore from nightly dump → run reconciliation → verify** `scripts/test_disaster_recovery.sh`	Disaster recovery exercise restoring staging database from backup and verifying ledger reconciliation passes. **Deliverable:** Disaster recovery test verified and documented (T-28 / AT-15).